(c) Marçal, J. 2009 - 2015

ESTRUTURA DE DADOS

ORDENAÇÃO DE DADOS

(c) Marçal, J. 2009 - 2015

ORDENAÇÃO E BUSCA

✓ O QUE PRECISAMOS SABER?

- PORQUE ORDENAR?
- QUANDO ORDENAR?
- O TEMPO GASTO NA ORDENAÇÃO PODE COMPROMETER A PERFORMANCE?
- QUAL ALGORITMO APLICAR?

(c) Marçal, J. 2009 - 2015

ORDENAÇÃO E BUSCA

✓ ALGORITMOS

- INSERTION SORT;
- SELECTION SORT;
- BUBBLE;
- SHELL;
- MERGE;
- HEAP;
- QUICK; QUICK3;
- OUTROS...

(c) Marçal, J. 2009 - 2015

ORDENAÇÃO E BUSCA

- ✓ **ALGORITMOS EXEMPLOS**
  - http://www.sorting-algorithms.com/
  - https://visualgo.net/en/sorting

(c) Marçal, J. 2009 - 2015

ORDENAÇÃO E BUSCA

✓ BUBBLE SORT

- ENTENDENDO (Demonstração no quadro);
- Consiste em realizar trocas sucessivas, trazendo para esquerda sempre o menor valor;
- Abaixo há um pequeno exemplo que continua, perceba que o primeiro elemento é comparado com o restante, logo depois o segundo elemento é comparado com o restante dos elementos, e sucessivamente...

2 | 2 | 18 | 12 | 18 |
2 é maior que **2** ?
Não, então continua a mesma coisa!

2 | 2 | 18 | 12 | 18 |
2 é maior que **18** ?
Não, então continua a mesma coisa!

2 | 2 | 18 | 12 | 18 |
18 é maior que **12** ?
Sim, então troca com o 12!

2 | 2 | 12 | 18 | 18 |
18 é maior que **18** ?
Não, então continua a mesma coisa!

(c) Marçal, J. 2009 - 2015

ORDENAÇÃO E BUSCA

✓ BUBBLE SORT

✓ ALGORITMO JAVA

```
for (int i = 1; i < lista.length; i++) {
    for (int j = 0; j < lista.length - 1; j++) {
        if (lista[j] > lista[j + 1]) {
            troca = lista[j + 1];
            lista[j + 1] = lista[j];
            lista[j] = troca;
        }
    }
}
```

(c) Marçal, J. 2009 - 2015

ORDENAÇÃO E BUSCA

✓ INSERTION SORT

- Inicia com no segundo valor, este é comparado com o primeiro, logo o terceiro elemento é comparado com os dois anteriores, o quarto com os três anteriores...
- ENTENDENDO (Exemplo no quadro);
- Temos a direita um exemplo didático;

Compara o **7** com o **7**
7 | 7 | 8 | 3 | 3 | 6

Compara o **8** com o **7, 7**
7 | 7 | 8 | 3 | 3 | 6

Compara o **3** com o **8, 7, 7**
7 | 7 | 8 | 3 | 3 | 6

Compara o **3** com o **8, 7, 7, 3**
3 | 7 | 7 | 8 | 3 | 6

Compara o **6** com o **8, 7, 7, 3, 3**
3 | 3 | 7 | 7 | 8 | 6

A ordem final é:
3 | 3 | 6 | 7 | 7 | 8

(c) Marçal, J. 2009 - 2015

ORDENAÇÃO E BUSCA

✓ INSERTION SORT

- CODIGO JAVA

```
public static void ordenaInsertion(){
    String vS = null;

    for (int i = 0; i < VET_NOMES.length; i++) {
        vS = VET_NOMES[i];

        for (int j = i - 1; (j >= 0) && (VET_NOMES[j].compareToIgnoreCase(vS) > 0); j--) {
            VET_NOMES[j + 1] = VET_NOMES[j];
            VET_NOMES[j] = vS;
        }
    }
}
```

(c) Marçal, J. 2009 - 2015

✓ MANIPULAÇÃO DE ARQUIVOS

- ESCREVER EM ARQUIVO TEXTO
  - FileWriter;
  - PrintWriter;
- LER ARQUIVO TEXTO
  - FileReader;
  - BufferedReader;

(c) Marçal, J. 2009 - 2015

ATIVIDADES

1. Carregue um vetor com as informações do arquivo cp.txt fornecido pelo professor;
2. Ordene o vetor utilizando o algoritmo **Bubble Sort**;
3. Salve as informações do vetor em um arquivo chamado cp-bubble.txt;
4. Repita o processo utilizando o algoritmo **Insertion Sort** e salve o arquivo com o nome cp-insertion.txt;
5. Compare os tempos de ordenação e diga qual o a ordenação que levou menos tempo para ser executada.